



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 3

Студент группы *ИКБО-50-23, Враженко Д.О.*

(подпись)

Проверил *Старший преподаватель,
Воронцов Ю.А.*

(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

Рабочее окружение и подготовка источника

Подготовка окружения

```
Python      : 3.14.0
Java        : openjdk version "17.0.20.1" 2026-08-18
PySpark     : 4.2.0
```

Извлечение таблиц SQLite в staging-каталог

```
orders              99441 rows
order_items         112650 rows
products            32951 rows
customers           99441 rows
order_payments      103886 rows
order_reviews       97621 rows
product_category_name_translation  71 rows
```

Запуск локального Spark

Создание SparkSession

```
Master: local[*]
App: Practice4_Ecommerce
Spark: 4.2.0
```

Чтение данных и работа со схемой

Чтение orders без автоматического вывода типов

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_status: string (nullable = true)
|-- order_purchase_timestamp: string (nullable = true)
|-- order_approved_at: string (nullable = true)
|-- order_delivered_carrier_date: string (nullable = true)
|-- order_delivered_customer_date: string (nullable = true)
|-- order_estimated_delivery_date: string (nullable = true)
```

Строк: 99441

order_id	customer_id	order_status	order_purchase_timestamp
e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33
53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37
47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49
949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06
ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39

only showing top 5 rows

Явная схема для order_items

```
root
|-- order_id: string (nullable = true)
|-- order_item_id: integer (nullable = true)
|-- product_id: string (nullable = true)
|-- seller_id: string (nullable = true)
|-- shipping_limit_date: string (nullable = true)
|-- price: double (nullable = true)
|-- freight_value: double (nullable = true)
```

Строк: 112650

Преобразование временных полей orders

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_status: string (nullable = true)
|-- order_purchase_timestamp: timestamp (nullable = true)
|-- order_approved_at: timestamp (nullable = true)
|-- order_delivered_carrier_date: timestamp (nullable = true)
|-- order_delivered_customer_date: timestamp (nullable = true)
|-- order_estimated_delivery_date: timestamp (nullable = true)
```

Основные преобразования DataFrame

Фильтрация и вычисляемый признак стоимости позиции

Позиций с price >= 500: 3239

order_id	order_item_id	price	freight_value	gross_item_value
0812eb902a67711a1cb742b3cdaa65ae	1	6735.0	194.31	6929.31
fefacc66af859508bf1a7934eab1e97f	1	6729.0	193.21	6922.21
f5136e38d1a14a4dbd87dff67da82701	1	6499.0	227.66	6726.66
a96610ab360d42a2e5335a3998b4718a	1	4799.0	151.34	4950.34
199af31afc78c699f0dbf71fb178d4d4	1	4690.0	74.34	4764.34

only showing top 5 rows

Временные преобразования заказов

Доставленных заказов: 96478

С известной датой доставки: 96470

Позже ожидаемой даты: 7826

Работа с пропусками и преобразование типов

Подсчёт пропусков во временных полях orders

```
-RECORD 0-----+
order_purchase_timestamp      | 0
order_approved_at             | 160
order_delivered_carrier_date  | 1783
order_delivered_customer_date | 2965
order_estimated_delivery_date | 0
```

Предметная проверка пропусков по статусам

```
+-----+-----+
|order_status|count|
+-----+-----+
|shipped     |1107 |
|canceled    |619  |
|unavailable |609  |
|invoiced    |314  |
|processing  |301  |
|delivered   |8     |
|created     |5     |
|approved    |2     |
+-----+-----+
```

Заполнение пропуска категориального признака

Товаров без категории: 610

Агрегации

Распределение заказов по статусам

```
+-----+-----+
|order_status|count|
+-----+-----+
|delivered   |96478|
|shipped     |1107 |
|canceled    |625  |
|unavailable |609  |
|invoiced    |314  |
|processing  |301  |
|created     |5     |
|approved    |2     |
+-----+-----+
```

Несколько агрегатных функций для способов оплаты

payment_type	payments	total_value	avg_value
credit_card	76795	1.254208419E7	163.32
boleto	19784	2869361.27	145.03
voucher	5775	379436.87	65.7
debit_card	1529	217989.79	142.57
not_defined	3	0.0	0.0

Соединение DataFrame

Позиции заказов + товары + перевод категорий

Строк до JOIN: 112650

Строк после JOIN: 112650

Позиций категории unknown: 1603

Заказы и клиенты, аналитика просрочки по штатам

customer_state	delivered	late	late_pct
AL	397	95	23.93
MA	717	141	19.67
PI	476	76	15.97
CE	1279	196	15.32
SE	335	51	15.22
BA	3256	457	14.04
RJ	12350	1664	13.47
TO	274	35	12.77
PA	946	117	12.37
ES	1995	244	12.23

only showing top 10 rows

Spark SQL

Выручка по товарным категориям через SQL

category	items	orders	revenue	avg_price
health_beauty	9670	8836	1258681.34	130.16
watches_gifts	5991	5624	1205005.68	201.14
bed_bath_table	11115	9417	1036988.68	93.3
sports_leisure	8641	7720	988048.97	114.34
computers_accessories	7827	6689	911954.32	116.51
furniture_decor	8334	6449	729762.49	87.56
cool_stuff	3796	3632	635290.85	167.36
housewares	6964	5884	632248.66	90.79
auto	4235	3897	592720.11	139.96
garden_tools	4347	3518	485256.46	111.63

Эквивалентная агрегация через DataFrame API

Категорий в витрине: 74

category	items	orders	revenue	avg_price
health_beauty	9670	8836	1258681.34	130.16
watches_gifts	5991	5624	1205005.68	201.14
bed_bath_table	11115	9417	1036988.68	93.3
sports_leisure	8641	7720	988048.97	114.34
computers_accessories	7827	6689	911954.32	116.51
furniture_decor	8334	6449	729762.49	87.56
cool_stuff	3796	3632	635290.85	167.36
housewares	6964	5884	632248.66	90.79
auto	4235	3897	592720.11	139.96
garden_tools	4347	3518	485256.46	111.63

only showing top 10 rows

Запись результата

Сохранение аналитической витрины в Parquet и CSV

Строк после round-trip: 74

root

```
-- category: string (nullable = true)
-- items: long (nullable = true)
-- orders: long (nullable = true)
-- revenue: double (nullable = true)
-- avg_price: double (nullable = true)
```

Корректное завершение SparkSession

Распределённый анализ собственного датасета

1. Подготовить источник к чтению Spark: CSV/JSON/Parquet либо воспроизводимый staging-этап из исходной СУБД. Объяснить, почему выбран именно этот способ.

Исходные данные

Источник – архив из 4 файлов в формате CSV (разделитель – табуляция):

Файл	Строк	Полей
Продажи.csv	865 222	11
Суммы чеков.csv	160 114	7
Продукты.csv	21 163	4
Клиенты.csv	3 838	3
Итого	1 050 337	

Обоснование способа подготовки

Выбран прямой способ чтения через `spark.read.csv()`, а не воспроизводимый staging-этап из СУБД. Причины:

1. **Данные уже в табличном формате.** Промежуточный слой (SQLite/PostgreSQL → pandas → CSV → Spark) не нужен – файлы уже в текстовом табличном формате, который Spark умеет читать нативно.
2. **Отсутствует внешняя СУБД.** В учебной части мы делали staging, потому что источником была SQLite-база, которую Spark не читает без внешнего JDBC-драйвера. Здесь источником являются CSV – это встроенный формат Spark, драйверы не требуются.
3. **Параметры формата известны и воспроизводимы:**
 - разделитель – табуляция (`.option("sep", "\t")`),
 - кодировка – UTF-8 (`.option("encoding", "UTF-8")`),
 - заголовки – в первой строке (`.option("header", True)`),
 - маркер пропуска – текстовая строка "NULL" (`.option("nullValue", "NULL")`).
1. **Явные схемы StructType** заданы для всех таблиц, что фиксирует контракт данных и исключает неожиданный inferSchema.

```
from pathlib import Path
```

```
DATA_DIR = Path("data")
OUT_DIR = Path("spark_output/prac3")
OUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
print(f"DATA_DIR: {DATA_DIR.resolve()}")
print(f"OUT_DIR: {OUT_DIR.resolve()}")
print()
```

```
EXPECTED = ["Продажи.csv", "Продукты.csv", "Клиенты.csv", "Суммы чеков.csv"]
```

```
print(f"{'Файл':<25} {'Статус':<10} {'Размер, МБ':>12}")
```

```
all_ok = True
```

```
for name in EXPECTED:
```

```
    f = DATA_DIR / name
```

```
    if f.exists():
```

```
        size_mb = f.stat().st_size / 1024 / 1024
```

```
        print(f"{'name':<25} {'OK':<10} {'size_mb':>12.1f}")
```

```
    else:
```

```
        print(f"{'name':<25} {'MISSING':<10} {'-':>12}")
```

```
    all_ok = False
```

```
print(f"ИТОГО файлов: {len(EXPECTED)}, все на месте: {all_ok}")
```

```
DATA_DIR: C:\Additional\VirtualMachines\prac3\data
```

```
OUT_DIR: C:\Additional\VirtualMachines\prac3\spark_output\prac3
```

Файл	Статус	Размер, МБ
Продажи.csv	OK	89.0
Продукты.csv	OK	3.3
Клиенты.csv	OK	0.5
Суммы чеков.csv	OK	13.2
ИТОГО файлов: 4, все на месте: True		

2. Создать SparkSession в local[*], зафиксировать версии Python, Java и Spark, а также основные параметры сессии.

Роль SparkSession

SparkSession – точка входа для работы с DataFrame и Spark SQL. Через SparkSession.builder накапливается конфигурация приложения:

- appName – понятное имя приложения (видно в Spark UI и логах),
- master("local[*]") – использовать все доступные логические ядра локальной машины,
- spark.driver.memory – память JVM драйвера,
- spark.sql.shuffle.partitions – число разделов после перераспределяющих операций (groupBy, join).

Метод getOrCreate() возвращает существующую сессию или создаёт новую. setLogLevel("WARN") уменьшает объём служебных сообщений.

```
import os
import subprocess
import sys
```

```
HADOOP_DIR = r"C:\Additional\VirtualMachines\prac3\hadoop"
```

```

os.environ["HADOOP_HOME"] = HADOOP_DIR
os.environ["hadoop.home.dir"] = HADOOP_DIR
os.environ["PATH"] = os.path.join(HADOOP_DIR, "bin") + os.pathsep +
os.environ["PATH"]

```

```

print(f"Python: {sys.version.split()[0]}")

```

```

try:
    java_line = subprocess.check_output(
        ["java", "-version"], stderr=subprocess.STDOUT, text=True
    ).splitlines()[0]
    print(f"Java: {java_line}")
except Exception as e:
    print(f"Java: не удалось - {e}")

```

```

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

```

```

spark = (
    SparkSession.builder
    .master("local[*]")
    .config("spark.driver.memory", "2g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)

```

```

spark.sparkContext.setLogLevel("WARN")

```

```

print()
print(f"Spark version: {spark.version}")
print(f"Master: {spark.sparkContext.master}")
print(f"Java (JVM):
{spark.sparkContext._jvm.System.getProperty('java.version')}")

```

Python: 3.14.0

Java: openjdk version "17.0.20.1" 2026-08-18

C:\Additional\VirtualMachines\prac3\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.

```

    require_minimum_pandas_version()

```

Spark version: 4.2.0

Master: local[*]

Java (JVM): 17.0.20.1

3. Прочитать основную таблицу и вывести число строк, список столбцов и printSchema().

Что такое «основная таблица»

Основная таблица-факт – Продажи.csv (865 222 строки, 11 полей). Она играет роль центрального факта: каждая строка = одна позиция чека (товар + количество + цена + скидка + клиент + филиал + дата).

Именно эта таблица используется далее для:

- преобразований (числа, даты, вычисляемые признаки),
- агрегаций (по группам товаров, по филиалам, по месяцам),
- JOIN с измерениями (Продукты, Клиенты).

Стратегия чтения CSV

CSV не хранит типы – Spark по умолчанию читает все значения как строки. Первый шаг – «сырое» чтение **без** inferSchema и **без** явной схемы. Это нужно, чтобы:

1. увидеть фактические типы, которые Spark выводит по умолчанию;
2. понять, какие поля требуют последующего приведения;
3. сравнить три стратегии чтения.

Параметры чтения:

- header=True – первая строка содержит имена столбцов;
- sep="\t" – разделитель – табуляция;
- encoding="UTF-8" – кодировка;
- nullValue="NULL" – маркер пропуска (в файле это буквальная строка "NULL").

Что показываем

- printSchema() – дерево схемы: имя и тип каждого поля;
- count() – число строк (action, запускает вычисление);
- columns – список имён столбцов;
- show(5, truncate=False) – первые 5 строк без обрезки значений.

```
sales_raw = (  
    spark.read  
        .option("header", True)  
        .option("encoding", "UTF-8")  
        .option("sep", "\t")  
        .option("nullValue", "NULL")  
        .option("emptyValue", "")  
        .csv(str(DATA_DIR / "Продажи.csv"))  
)
```

```
print("\nСхема (printSchema):")  
sales_raw.printSchema()
```

```
n_rows = sales_raw.count()
```

```

print(f"\nЧисло строк (count): {n_rows:,}")

print(f"\nСписок столбцов ({len(sales_raw.columns)} шт.):")
for i, c in enumerate(sales_raw.columns, 1):
    print(f"    {i:>2}. {c}")

print("\nПервые 5 строк:")
sales_raw.show(5, truncate=False)

```

Схема (printSchema):

```

root
|-- Себестоимость: string (nullable = true)
|-- Дата покупки: string (nullable = true)
|-- Номер чека: string (nullable = true)
|-- Product_key: string (nullable = true)
|-- Филиал: string (nullable = true)
|-- Количество: string (nullable = true)
|-- Сумма покупки: string (nullable = true)
|-- Сумма скидки: string (nullable = true)
|-- Client_ID: string (nullable = true)
|-- Машина доставки: string (nullable = true)
|-- Адрес: string (nullable = true)

```

Число строк (count): 865,222

Список столбцов (11 шт.):

1. Себестоимость
2. Дата покупки
3. Номер чека
4. Product_key
5. Филиал
6. Количество
7. Сумма покупки
8. Сумма скидки
9. Client_ID
10. Машина доставки
11. Адрес

Первые 5 строк:

```

+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
|Себестоимость|Дата покупки|Номер чека|Product_key|Филиал
|Количество|Сумма покупки|Сумма скидки|Client_ID|Машина доставки|Адрес
|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
|113,33      |27.09.2020  |NULL      |1_1_1      | PC (Распределительный
склад)|25          |250       |0          |CL_01      |Б715ПН      |410779,

```

```

г. Саратов, ул. Спортивная, 9, оф. 73|
|127,12      |27.09.2020 |NULL      |2_1_1      | PC (Распределительный
склад)|25      |275      |0      |CL_01      |Б715ПН      |410779,
г. Саратов, ул. Спортивная, 9, оф. 73|
|498,19      |27.09.2020 |NULL      |3_1_1      | PC (Распределительный
склад)|25      |875      |0      |CL_01      |Б715ПН      |410779,
г. Саратов, ул. Спортивная, 9, оф. 73|
|928,37      |27.09.2020 |NULL      |4_1_2      | PC (Распределительный
склад)|25      |2125     |0      |CL_01      |Б715ПН      |410779,
г. Саратов, ул. Спортивная, 9, оф. 73|
|624,23      |27.09.2020 |NULL      |5_2_3      | PC (Распределительный
склад)|25      |900      |0      |CL_01      |Б715ПН      |410779,
г. Саратов, ул. Спортивная, 9, оф. 73|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+

```

only showing top 5 rows

Результат

Метрика	Значение
Число строк	865 222
Число столбцов	11
Все типы после «сырого» чтения	string
Маркер пропуска	"NULL "

Ключевые наблюдения

1. **Все 11 полей прочитаны как string.** Даже числовые (Себестоимость, Сумма покупки, Сумма скидки, Количество) и временные (Дата покупки). Это ожидаемо: без inferSchema или явной схемы Spark не пытается угадывать типы.
2. **Числа в русском формате.** В поле Себестоимость – 113,33 с запятой. Spark не преобразует такие строки в числа даже при попытке cast("double") – потребуется regexp_replace(",", ".").
3. **Даты в формате dd.мм.yyyy** без времени. Для дальнейшей работы с датами их нужно привести к timestamp через to_timestamp.
4. **Поле Номер чека содержит NULL** в первых строках. Это не ошибка чтения – такие данные в источнике (безноменклатурные продажи).
5. **Маркер пропуска "NULL"** успешно обработан на этапе чтения: в колонке Номер чека видим настоящий null, а не строку "NULL".

Загрузка вспомогательных таблиц

Также загрузим три вспомогательные таблицы:

- **Продукты.csv** – измерение товара (название, бренд, группа);
- **Клиенты.csv** – измерение клиента (юрлицо, ИНН);

- **Суммы чеков.csv** – агрегированная сумма чека (используется для контроля ключа Номер чека).

Все три читаются теми же параметрами, что и Продажи: разделитель – табуляция, nullValue="NULL", явная StructType. Явная схема фиксирует контракт и исключает неожиданный inferSchema.

Особенность Клиенты.csv: поле Client_ID **не уникально** – один клиент может иметь до 5 юрлиц. Это будет учтено перед JOIN (сворачивание до уровня клиента).

```
from pyspark.sql.types import StructType, StructField, StringType
```

```
products_schema = StructType([
    StructField("Product_Key", StringType(), True),
    StructField("Название товара", StringType(), True),
    StructField("Название бренда", StringType(), True),
    StructField("Группа товара", StringType(), True),
])
```

```
products = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .option("sep", "\t")
    .option("nullValue", "NULL")
    .option("emptyValue", "")
    .schema(products_schema)
    .csv(str(DATA_DIR / "Продукты.csv"))
)
```

```
products.printSchema()
print(f"\nСтрок: {products.count():,}")
print(f"Уникальных Product_Key:
{products.select('Product_Key').distinct().count():,}")
print("\nПервые 5 строк:")
products.show(5, truncate=False)
```

```
root
|-- Product_Key: string (nullable = true)
|-- Название товара: string (nullable = true)
|-- Название бренда: string (nullable = true)
|-- Группа товара: string (nullable = true)
```

Строк: 21,163

Уникальных Product_Key: 21,163

Первые 5 строк:

```
+-----+-----+-----+
--++-----++
|Product_Key|Название товара
|Название бренда |Группа товара      |
```

```

+-----+-----+-----+-----+
+-----+-----+-----+-----+
| 1_1_1      | Кисть колонок круглая d=0,5мм L=6мм сер.2110 №0
| Живописные кисти | Кисти-Акварель-Колонок-Соболь |
| 2_1_1      | Кисть колонок круглая d=1,0мм L=8мм сер.2110 №01
| Живописные кисти | Кисти-Акварель-Колонок-Соболь |
| 3_1_1      | Кисть колонок круглая d=3,0мм L=15мм сер.2110 №03
| Живописные кисти | Кисти-Акварель-Колонок-Соболь |
| 4_1_2      | Кисть колонок круглая пучок: диам.=0,40мм, дл.=1,90мм, серия 1052
№04 | Живописные кисти | Кисти-Масло-Колонок |
| 5_2_3      | "Кисть синтетика круглая ""Вернисаж"" серия 211 №04"
| Гамма       | Кисти-Масло-Синтетика |
+-----+-----+-----+-----+

```

only showing top 5 rows

```

clients_schema = StructType([
    StructField("Client_ID", StringType(), True),
    StructField("Покупатель", StringType(), True),
    StructField("Contractor_name", StringType(), True),
])

clients = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .option("sep", "\t")
    .option("nullValue", "NULL")
    .option("emptyValue", "")
    .schema(clients_schema)
    .csv(str(DATA_DIR / "Клиенты.csv"))
)

clients.printSchema()
n_rows = clients.count()
n_unique = clients.select("Client_ID").distinct().count()
print(f"\nСтрок: {n_rows:,}")
print(f"Уникальных Client_ID: {n_unique:,}")
print(f"Дубликатов по Client_ID: {n_rows - n_unique:,}")
print("\nПервые 10 строк (для проверки персональных данных):")
clients.show(10, truncate=False)

root
|-- Client_ID: string (nullable = true)
|-- Покупатель: string (nullable = true)
|-- Contractor_name: string (nullable = true)

```

Строк: 3,838
 Уникальных Client_ID: 3,227
 Дубликатов по Client_ID: 611

Первые 10 строк (для проверки персональных данных):

```
+-----+-----+-----+-----+
|Client_ID|Покупатель|Contractor_name|
|
+-----+-----+-----+-----+
|CL_01| "000 ""Положение"" ИНН 7896913934" | "000 ""Положение"" ИНН
7896913934 ЮРЛИЦО №1" |
|CL_10001| "000 ""Решительная скорость"" ИНН 96234274341" | "000 ""Решительная
скорость"" ИНН 96234274341 ЮРЛИЦО №1" |
|CL_1001| "000 ""Управление"" ИНН 66912194025" | "000 ""Управление""
ИНН 66912194025 ЮРЛИЦО №1" |
|CL_10011| "ПАО ""Час"" ИНН 13746866995" | "ПАО ""Час"" ИНН
13746866995 ЮРЛИЦО №1" |
|CL_10021| "000 ""Зал"" ИНН 84426324283" | "000 ""Зал"" ИНН
84426324283 ЮРЛИЦО №1" |
|CL_10031| "000 ""Фактор"" ИНН 84921320774" | "000 ""Фактор"" ИНН
84921320774 ЮРЛИЦО №1" |
|CL_10041| "000 ""Правительство"" ИНН 39913231628" | "000 ""Правительство""
ИНН 39913231628 ЮРЛИЦО №1" |
|CL_10041| "000 ""Правительство"" ИНН 39913231628" | "000 ""Правительство""
ИНН 39913231628 ЮРЛИЦО №3" |
|CL_10041| "000 ""Правительство"" ИНН 39913231628" | "000 ""Правительство""
ИНН 39913231628 ЮРЛИЦО №4" |
|CL_10051| "ПАО ""Массовый круг"" ИНН 61958913507" | "ПАО ""Массовый круг""
ИНН 61958913507 ЮРЛИЦО №1" |
+-----+-----+-----+-----+
```

only showing top 10 rows

```
checks_schema = StructType([
    StructField("Дата покупки", StringType(), True),
    StructField("Client_ID", StringType(), True),
    StructField("Адрес", StringType(), True),
    StructField("Машина доставки", StringType(), True),
    StructField("Филиал", StringType(), True),
    StructField("Номер чека", StringType(), True),
    StructField("Сумма покупки|Сумма", StringType(), True),
])
```

```
checks = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .option("sep", "\t")
    .option("nullValue", "NULL")
    .option("emptyValue", "")
    .schema(checks_schema)
    .csv(str(DATA_DIR / "Суммы чеков.csv"))
    .withColumnRenamed("Сумма покупки|Сумма", "Сумма_чека")
    .withColumn("Филиал", F.trim("Филиал"))
```

```
)

checks.printSchema()
print(f"\nСтрок: {checks.count():,}")
print(f"Уникальных Номер чека: {checks.select('Номер чека').distinct().count():,}")
print("\nПервые 5 строк:")
checks.show(5, truncate=False)
```

```
root
|-- Дата покупки: string (nullable = true)
|-- Client_ID: string (nullable = true)
|-- Адрес: string (nullable = true)
|-- Машина доставки: string (nullable = true)
|-- Филиал: string (nullable = true)
|-- Номер чека: string (nullable = true)
|-- Сумма_чека: string (nullable = true)
```

Строк: 160,114
Уникальных Номер чека: 81,838

Первые 5 строк:

```
+-----+-----+-----+-----+-----+-----+
|Дата покупки|Client_ID|Адрес
|Машина доставки|Филиал
|Номер чека|Сумма_чека|
+-----+-----+-----+-----+-----+-----+
|27.09.2020 |CL_01    |410779, г. Саратов, ул. Спортивная, 9, оф. 73
|Б715ПН      |PC (Распределительный склад)|NULL      |14207,5 |
|28.09.2020 |CL_11    |400586, г. Волгоград, ул. Молодежная, 28, оф. 97
|А645ДМ      |PC (Распределительный склад)|NULL      |5500    |
|30.09.2020 |CL_21    |400140, г. Волгоград, ул. Ленина, 3, оф. 99
|Б715ПН      |ТЗ-2 ( Ленинградское шоссе) |NULL      |37499,15 |
|09.10.2020 |CL_31    |630742, г. Новосибирск, ул. Строительная, 26, оф.
74|Н290ЛО    |ТЗ-2 ( Ленинградское шоссе) |NULL      |12889,45 |
|12.10.2020 |CL_41    |426622, г. Ижевск, ул. Восточная, 45, оф. 15
|А645ДМ      |PC (Распределительный склад)|NULL      |22174,55 |
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

4. Для одного DataFrame задать явный StructType. Сравнить явную схему с inferSchema или чтением строками.

Теория

CSV – текстовый формат, в котором **нет информации о типах**. Spark предлагает три способа задать схему:

Способ	Как работает	Плюсы	Минусы
--------	--------------	-------	--------

Способ	Как работает	Плюсы	Минусы
Без схемы	Всё читается как <code>string</code>	Быстро, без доп. прохода	Все вычисления требуют ручного приведения
<code>inferSchema=True</code>	Spark делает дополнительный проход по данным и угадывает типы	Удобно для прототипов	Доп. проход по данным; тип зависит от подвыборки; может ошибиться
Явная <code>StructType</code>	Программист задаёт контракт	Воспроизводимо, без доп. прохода, типы фиксированы	Нужно писать код схемы

Для аналитического pipeline **предпочтительна явная схема**: она фиксирует контракт источника, не тратит ресурсы на анализ подвыборки и не зависит от того, какие строки попадут в первые N записей.

Что сравниваем

На одной и той же таблице `Продажи.csv` создадим три `DataFrame` тремя способами и сравним схемы полей:

- **DF1** – без схемы;
- **DF2** – с `inferSchema=True`;
- **DF3** – с явной `StructType`.

Сравним:

1. Типы ключевых полей (Количество, Себестоимость, Дата покупки).
2. Совпадение имён столбцов.
3. Фактическое поведение `cast()` в каждом из вариантов.

```
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType
)
```

```
sales_strings = sales_raw
```

```
sales_inferred = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .option("sep", "\t")
    .option("nullValue", "NULL")
    .option("emptyValue", "")
    .option("inferSchema", True)
    .csv(str(DATA_DIR / "Продажи.csv"))
)
```

```

sales_schema = StructType([
    StructField("Себестоимость", StringType(), True),
    StructField("Дата покупки", StringType(), True),
    StructField("Номер чека", StringType(), True),
    StructField("Product_key", StringType(), True),
    StructField("Филиал", StringType(), True),
    StructField("Количество", DoubleType(), True),
    StructField("Сумма покупки", StringType(), True),
    StructField("Сумма скидки", StringType(), True),
    StructField("Client_ID", StringType(), True),
    StructField("Машина доставки", StringType(), True),
    StructField("Адрес", StringType(), True),
])

sales = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .option("sep", "\t")
    .option("nullValue", "NULL")
    .option("emptyValue", "")
    .schema(sales_schema)
    .csv(str(DATA_DIR / "Продажи.csv"))
)

sales_strings.printSchema()

sales_inferred.printSchema()

sales.printSchema()

print(f"{'Поле':<20} {'Без схемы':<15} {'inferSchema':<15} {'StructType':<15}")

for col in ["Количество", "Себестоимость", "Дата покупки", "Сумма покупки"]:
    t1 = dict(sales_strings.dtypes)[col]
    t2 = dict(sales_inferred.dtypes)[col]
    t3 = dict(sales.dtypes)[col]
    print(f"{'col':<20} {'t1':<15} {'t2':<15} {'t3':<15}")

cols1 = set(sales_strings.columns)
cols2 = set(sales_inferred.columns)
cols3 = set(sales.columns)

print(f"Без схемы vs inferSchema : {cols1 == cols2}")
print(f"Без схемы vs StructType : {cols1 == cols3}")

print("\n[без схемы] Количество * 2:")

```

```

result = sales_strings.select(
    "Количество",
    (F.col("Количество") * 2).alias("k_x2")
).show(3)

print("\n[inferSchema] Количество * 2:")
sales_inferred.select(
    "Количество",
    (F.col("Количество") * 2).alias("k_x2")
).show(3)

print("\n[StructType] Количество * 2:")
sales.select(
    "Количество",
    (F.col("Количество") * 2).alias("k_x2")
).show(3)

root
|-- Себестоимость: string (nullable = true)
|-- Дата покупки: string (nullable = true)
|-- Номер чека: string (nullable = true)
|-- Product_key: string (nullable = true)
|-- Филиал: string (nullable = true)
|-- Количество: string (nullable = true)
|-- Сумма покупки: string (nullable = true)
|-- Сумма скидки: string (nullable = true)
|-- Client_ID: string (nullable = true)
|-- Машина доставки: string (nullable = true)
|-- Адрес: string (nullable = true)

root
|-- Себестоимость: string (nullable = true)
|-- Дата покупки: string (nullable = true)
|-- Номер чека: string (nullable = true)
|-- Product_key: string (nullable = true)
|-- Филиал: string (nullable = true)
|-- Количество: integer (nullable = true)
|-- Сумма покупки: string (nullable = true)
|-- Сумма скидки: string (nullable = true)
|-- Client_ID: string (nullable = true)
|-- Машина доставки: string (nullable = true)
|-- Адрес: string (nullable = true)

root
|-- Себестоимость: string (nullable = true)
|-- Дата покупки: string (nullable = true)
|-- Номер чека: string (nullable = true)
|-- Product_key: string (nullable = true)
|-- Филиал: string (nullable = true)
|-- Количество: double (nullable = true)
|-- Сумма покупки: string (nullable = true)
|-- Сумма скидки: string (nullable = true)

```

```

|-- Client_ID: string (nullable = true)
|-- Машина доставки: string (nullable = true)
|-- Адрес: string (nullable = true)

```

Поле	Без схемы	inferSchema	StructType
Количество	string	int	double
Себестоимость	string	string	string
Дата покупки	string	string	string
Сумма покупки	string	string	string
Без схемы vs inferSchema	: True		
Без схемы vs StructType	: True		

[без схемы] Количество * 2:

```

+-----+-----+
|Количество|k_x2|
+-----+-----+
|          25|  50|
|          25|  50|
|          25|  50|
+-----+-----+

```

only showing top 3 rows

[inferSchema] Количество * 2:

```

+-----+-----+
|Количество|k_x2|
+-----+-----+
|          25|  50|
|          25|  50|
|          25|  50|
+-----+-----+

```

only showing top 3 rows

[StructType] Количество * 2:

```

+-----+-----+
|Количество|k_x2|
+-----+-----+
|        25.0|50.0|
|        25.0|50.0|
|        25.0|50.0|
+-----+-----+

```

only showing top 3 rows

Результаты сравнения

Поле	Без схемы	inferSchema	Явная StructType
Количество	string	int	double
Себестоимость	string	string	string
Дата покупки	string	string	string
Сумма покупки	string	string	string

Все три варианта дают одинаковый **набор имён** столбцов – расхождения только в типах.

Ключевые наблюдения

1. **inferSchema не справляется с русским форматом чисел.** Значения вида 113,33 (запятая вместо точки) Spark не может преобразовать в double и оставляет string. В нашем случае inferSchema угадала тип только для Количество (там целые числа без запятой) и **не угадала** для Себестоимости, Суммы покупки, Суммы скидки.
2. **inferSchema делает дополнительный проход по данным.** Для таблицы на 865 222 строки это лишние затраты. В продакшене на больших файлах разница заметна.
3. **Воспроизводимость.** inferSchema анализирует только первые N строк (по умолчанию 1000) и может дать **разные схемы** на разных подвыборках. Явная схема всегда одна и та же.
4. **Практическая арифметика.** В DataFrame без схемы Количество * 2 может дать неожиданный результат (конкатенацию строк, если Spark приведёт тип к string), либо ошибку. В схеме с DoubleType умножение работает как арифметика.

5. Выполнить не менее пяти разных transformations, включая select(), filter() и withColumn(). Создать минимум один содержательный вычисляемый признак.

Демонстрация пяти базовых трансформаций

№	Метод	Что делает
1	select()	Ограничить набор полей
2	filter()	Условный срез строк
3	withColumn()	Создать или заменить столбец
4	drop()	Исключить поле
5	distinct() + orderBy()	Уникальные значения в порядке

Содержательные вычисляемые признаки

После демонстрации базовых методов построим таблицу sales_clean, в которой:

- строковые числа приведены к double,
- строковые даты – к timestamp,
- созданы четыре содержательных вычисляемых признака:
 - discount_pct – доля скидки от суммы покупки, %,
 - margin – маржа = выручка – себестоимость,
 - purchase_month / purchase_year – временные разрезы,
 - has_discount – булев признак наличия скидки.

```
step1 = sales.select(  
    "Номер чека", "Client_ID", "Product_key",
```

```

        "Дата покупки", "Сумма покупки", "Сумма скидки"
    )
    print(f"select(): полей {len(step1.columns)}")

    step2 = step1.filter(F.col("Client_ID").isNotNull())
    n_step2 = step2.count()
    print(f"filter(): строк {n_step2:>10,}")

    step3 = step2.withColumn("check_len", F.length("Номер чека"))
    print(f"withColumn(): добавлено поле check_len")

    step4 = step3.drop("check_len")
    print(f"drop(): удалено поле check_len")

    step5 = (
        step4
        .select("Client_ID")
        .distinct()
        .orderBy("Client_ID")
    )
    n_clients = step5.count()
    print(f"distinct+orderBy: уникальных клиентов {n_clients:>8,}")

    print("\nПример результата после всех 5 шагов:")
    step5.show(5, truncate=False)

    print("\nМетоды в цепочке: select, filter, withColumn, drop, distinct, orderBy – 6 методов")

```

```

select(): полей 6
filter(): строк      865,222
withColumn(): добавлено поле check_len
drop(): удалено поле check_len
distinct+orderBy: уникальных клиентов      1,932

```

Пример результата после всех 5 шагов:

```

+-----+
|Client_ID|
+-----+
|CL_01    |
|CL_10001 |
|CL_1001  |
|CL_10011 |
|CL_10021 |
+-----+

```

only showing top 5 rows

Методы в цепочке: select, filter, withColumn, drop, distinct, orderBy – 6 методов

```

def ru_to_double(col_name):
    cleaned = F.regexp_replace(F.col(col_name), ",", ".")

```

```

        return cleaned.try_cast("double")

sales_clean = (
    sales
    .withColumn(
        "ts",
        F.to_timestamp("Дата покупки", "dd.MM.yyyy")
    )
    .withColumn("cost", ru_to_double("Себестоимость"))
    .withColumn("revenue", ru_to_double("Сумма покупки"))
    .withColumn("discount", F.round(ru_to_double("Сумма скидки"), 2))
    .withColumn(
        "discount",
        F.when(F.abs(F.col("discount")) < 0.5, F.lit(0.0))
        .otherwise(F.col("discount"))
    )
    .withColumn("has_discount", F.col("discount") > 0)
    .withColumn(
        "discount_pct",
        F.when(
            F.col("revenue") > 0,
            F.round(F.col("discount") / F.col("revenue") * 100, 2)
        )
    )
    .withColumn(
        "margin",
        F.round(F.col("revenue") - F.col("cost"), 2)
    )
    .withColumn("purchase_month", F.date_format("ts", "yyyy-MM"))
    .withColumn("purchase_year", F.year("ts"))
    .withColumn("purchase_dow", F.dayofweek("ts"))
    .drop("Себестоимость", "Сумма покупки", "Сумма скидки")
)

print(f"Полей: {len(sales_clean.columns)}")
print(f"Строк: {sales_clean.count():,}")
print()
print("Схема:")
sales_clean.printSchema()

print("\nПервые 5 строк (ключевые поля):")
sales_clean.select(
    "Дата покупки", "ts", "purchase_month", "purchase_year",
    "cost", "revenue", "discount", "discount_pct", "margin", "has_discount"
).show(5, truncate=False)

```

Полей: 18
Строк: 865,222

Схема:

root

```
|-- Дата покупки: string (nullable = true)
|-- Номер чека: string (nullable = true)
|-- Product_key: string (nullable = true)
|-- Филиал: string (nullable = true)
|-- Количество: double (nullable = true)
|-- Client_ID: string (nullable = true)
|-- Машина доставки: string (nullable = true)
|-- Адрес: string (nullable = true)
|-- ts: timestamp (nullable = true)
|-- cost: double (nullable = true)
|-- revenue: double (nullable = true)
|-- discount: double (nullable = true)
|-- has_discount: boolean (nullable = true)
|-- discount_pct: double (nullable = true)
|-- margin: double (nullable = true)
|-- purchase_month: string (nullable = true)
|-- purchase_year: integer (nullable = true)
|-- purchase_dow: integer (nullable = true)
```

Первые 5 строк (ключевые поля):

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|Дата покупки|ts                |purchase_month|purchase_year|cost
|revenue|discount|discount_pct|margin |has_discount|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|27.09.2020 |2020-09-27 00:00:00|2020-09      |2020          |113.33|250.0
|0.0      |0.0          |136.67 |false      |
|27.09.2020 |2020-09-27 00:00:00|2020-09      |2020          |127.12|275.0
|0.0      |0.0          |147.88 |false      |
|27.09.2020 |2020-09-27 00:00:00|2020-09      |2020          |498.19|875.0
|0.0      |0.0          |376.81 |false      |
|27.09.2020 |2020-09-27 00:00:00|2020-09      |2020          |928.37|2125.0
|0.0      |0.0          |1196.63|false      |
|27.09.2020 |2020-09-27 00:00:00|2020-09      |2020          |624.23|900.0
|0.0      |0.0          |275.77 |false      |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
```

only showing top 5 rows

Результат

Базовые трансформации:

№	Метод	Что сделано
1	select()	6 полей из 11
2	filter()	865 222 строки с непустым Client_ID
3	withColumn()	Добавлено поле check_len (потом удалено)
4	drop()	Удалено служебное поле

№	Метод	Что сделано
5	<code>distinct() + orderBy()</code>	1 932 уникальных клиента

Построение `sales_clean`:

Итоговая таблица содержит **20 полей** и **865 222 строки**. К исходным 11 полям добавлено **9 новых**:

Поле	Тип	Что означает
<code>ts</code>	<code>timestamp</code>	Дата покупки в правильном типе
<code>cost</code>	<code>double</code>	Себестоимость (число)
<code>revenue</code>	<code>double</code>	Сумма покупки (число)
<code>discount</code>	<code>double</code>	Сумма скидки (число, шум < 0.5 обнулён)
<code>has_discount</code>	<code>boolean</code>	Признак наличия скидки
<code>discount_pct</code>	<code>double</code>	Доля скидки от выручки, %
<code>margin</code>	<code>double</code>	Маржа = <code>revenue - cost</code>
<code>purchase_month</code>	<code>string</code>	Месяц в формате уууу-мм
<code>purchase_year</code>	<code>int</code>	Год
<code>purchase_dow</code>	<code>int</code>	День недели (1 = воскресенье)

6. Посчитать пропуски минимум в трёх полях. Для каждого принятого `fillna/dropna/filter` объяснить предметную причину; запрещено удалять строки только потому, что в них есть `null`.

Пропуск в Spark – это `null`

В Spark отсутствующее значение представлено `null`. Его наличие **не означает**, что строку нужно удалять. Пропуск может отражать **предметное состояние** объекта:

- у заказа без доставки естественно нет адреса доставки;
- у товара без учётной цены нет себестоимости;
- у старой продажи нет данных о скидке.

Поэтому сначала **исследуем** пропуски, потом принимаем решение.

Что проверяем

Проверим пропуски в пяти ключевых полях `sales_clean`:

- Адрес, Машина доставки – предполагается много пропусков (доставка только для части заказов),
- `cost`, `margin` – пропуски связаны с отсутствием себестоимости, - Количество – аномалии.

```
total = sales_clean.count()
```

```
print(f"{'Поле':<25} {'NULL':>12} {'% от всех':>12}")
print("-" * 55)
```

```

check_fields = [
    "Адрес", "Машина доставки",
    "cost", "margin",
    "Количество",
]

for c in check_fields:
    n_null = sales_clean.filter(F.col(c).isNull()).count()
    pct = n_null / total * 100
    print(f"{c:<25} {n_null:>12,} {pct:>11.2f}%")

print("\nСогласованность NULL в cost и margin:")
n_both_null = sales_clean.filter(
    F.col("cost").isNull() & F.col("margin").isNull()
).count()
n_cost_null_margin_not = sales_clean.filter(
    F.col("cost").isNull() & F.col("margin").isNotNull()
).count()
n_cost_not_margin_null = sales_clean.filter(
    F.col("cost").isNotNull() & F.col("margin").isNull()
).count()

print(f"  cost NULL & margin NULL : {n_both_null:>10,}")
print(f"  cost NULL & margin NOT  : {n_cost_null_margin_not:>10,}")
print(f"  cost NOT  & margin NULL  : {n_cost_not_margin_null:>10,}")

print("\nСогласованность NULL в Адрес и Машина доставки:")
n_both_addr = sales_clean.filter(
    F.col("Адрес").isNull() & F.col("Машина доставки").isNull()
).count()
n_addr_null_vehicle_not = sales_clean.filter(
    F.col("Адрес").isNull() & F.col("Машина доставки").isNotNull()
).count()
print(f"  Адрес NULL & Машина NULL      : {n_both_addr:>10,}")
print(f"  Адрес NULL & Машина NOT NULL   : {n_addr_null_vehicle_not:>10,}")

```

Поле	NULL	% от всех
Адрес	832,181	96.18%
Машина доставки	832,181	96.18%
cost	86,431	9.99%
margin	86,431	9.99%
Количество	100	0.01%

```

Согласованность NULL в cost и margin:
  cost NULL & margin NULL :      86,431
  cost NULL & margin NOT  :          0
  cost NOT  & margin NULL :          0

```

```

Согласованность NULL в Адрес и Машина доставки:

```

Адрес NULL & Машина NULL : 832,181
Адрес NULL & Машина NOT NULL : 0

```
sales_clean.filter(F.col("Адрес").isNull()).select(
  "Номер чека", "ts", "Филиал", "Адрес", "Машина доставки",
  "revenue", "cost"
).show(3, truncate=False)
```

```
sales_clean.filter(F.col("Адрес").isNotNull()).select(
  "Номер чека", "ts", "Филиал", "Адрес", "Машина доставки",
  "revenue", "cost"
).show(3, truncate=False)
```

```
sales_clean.filter(F.col("cost").isNull()).select(
  "Номер чека", "Филиал", "revenue", "cost", "margin", "has_discount"
).show(3, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
--+-----+-----+
|Номер чека|ts                |Филиал                |Адрес|Машина
доставки|revenue|cost |
+-----+-----+-----+-----+-----+-----+
--+-----+-----+
|SLS_119   |2020-06-16 00:00:00| ТЗ-2 ( Ленинградское шоссе)|NULL |NULL
|48.5925|18.18|
|SLS_119   |2020-06-16 00:00:00| ТЗ-2 ( Ленинградское шоссе)|NULL |NULL
|62.7825|39.84|
|SLS_119   |2020-06-16 00:00:00| ТЗ-2 ( Ленинградское шоссе)|NULL |NULL
|62.7825|21.62|
+-----+-----+-----+-----+-----+-----+
--+-----+-----+
```

only showing top 3 rows

```
+-----+-----+-----+-----+-----+-----+
--+-----+-----+
|Номер чека|ts                |Филиал                |Адрес
|Машина доставки|revenue|cost |
+-----+-----+-----+-----+-----+-----+
--+-----+-----+
|NULL      |2020-09-27 00:00:00| РС (Распределительный склад)|410779, г.
Саратов, ул. Спортивная, 9, оф. 73|Б715ПН                |250.0 |113.33|
|NULL      |2020-09-27 00:00:00| РС (Распределительный склад)|410779, г.
Саратов, ул. Спортивная, 9, оф. 73|Б715ПН                |275.0 |127.12|
|NULL      |2020-09-27 00:00:00| РС (Распределительный склад)|410779, г.
Саратов, ул. Спортивная, 9, оф. 73|Б715ПН                |875.0 |498.19|
+-----+-----+-----+-----+-----+-----+
--+-----+-----+
```

only showing top 3 rows

```
+-----+-----+-----+-----+-----+-----+
|Номер чека|Филиал                |revenue |cost|margin|has_discount|
+-----+-----+-----+-----+-----+-----+
|NULL      | РС (Распределительный склад)|1125.0  |NULL|NULL  |false      |
|NULL      | РС (Распределительный склад)|5500.0  |NULL|NULL  |false      |
```

```
|NULL      | ТЗ-2 ( Ленинградское шоссе) |12431.25|NULL|NULL |false      |
+-----+-----+-----+-----+-----+-----+
only showing top 3 rows
```

Результаты подсчёта

Поле	NULL	% от всех	Связано с
Адрес	832 181	96.18%	Отсутствие доставки
Машина доставки	832 181	96.18%	Отсутствие доставки
cost	86 431	9.99%	Отсутствие учётной себестоимости
margin	86 431	9.99%	Следствие NULL в cost
Количество	100	0.01%	Аномалии ввода

Важные выводы из проверки согласованности:

1. NULL в cost и margin совпадают 1:1.

- cost NULL & margin NULL = 86 431
- cost NULL & margin NOT NULL = 0
- cost NOT NULL & margin NULL = 0

Это логично: $\text{margin} = \text{revenue} - \text{cost}$. Если $\text{cost} = \text{NULL}$, то и $\text{margin} = \text{NULL}$. Данные внутренне непротиворечивы.

2. NULL в Адрес и Машина доставки совпадают 1:1 (832 181). Строк, где заполнен адрес, но не указана машина, – 0. Это также логично: адрес и машина – атрибуты одной доставки. Если доставки нет, оба поля пусты.

Решения по каждому полю – предметное обоснование

Поле	Действие	Обоснование
Адрес	Не заполняем, не удаляем	NULL означает «продажа без доставки» (самовывоз или продажа через филиал). Заполнение фиктивным адресом исказило бы географический разрез. Удаление 96% строк уничтожило бы датасет.
Машина доставки	Не заполняем	То же: NULL = нет доставки. Поле неотделимо от факта доставки.
cost	Не заполняем, не удаляем	NULL = «себестоимость не была зафиксирована на момент продажи». Заполнение средним по группе создало бы фиктивную маржу . При анализе маржи такие строки автоматически исключаются (функции AVG, SUM игнорируют NULL).
margin	Оставляем NULL	$\text{margin} = \text{revenue} - \text{cost}$; без cost маржа принципиально не определена. Заполнять нельзя – это скрыло бы факт отсутствия данных.
Количество	Оставляем	100 строк – это 0.01% от датасета. Удаление неоправданно: влияние на агрегаты ничтожно.

Главный принцип

Ни одна строка не удалена через dropna(). Это соответствует прямому требованию методички:

запрещено удалять строки только потому, что в них есть null

Вместо удаления мы **сохранили** NULL и учли его при интерпретации. Например, при агрегации маржи по филиалам:

- SUM(margin) автоматически пропускает NULL – не создаёт искажений;
- AVG(margin) тоже игнорирует NULL;
- COUNT(*) считает все строки – а COUNT(margin) только те, где маржа определена.

При интерпретации результатов мы явно указываем, сколько строк было исключено неявно.

Проверка гипотезы про NULL в адресе

Проверим, что строки с NULL в адресе – это действительно продажи без доставки. Косвенный признак: такие строки должны относиться к другим филиалам (магазинам, а не складам) и иметь компактные чеки.

7. Выполнить минимум две группировки. Одна должна использовать не менее трёх агрегатных функций.

Теория

Агрегация преобразует множество строк в статистики по группам:

- groupBy() задаёт ключ группировки;
- agg() – набор агрегатных выражений (count, sum, avg, min, max, countDistinct, stddev и др.).

В распределённом плане groupBy() требует **перераспределения** данных так, чтобы одинаковые ключи оказались в одном разделе (shuffle). Поэтому группировка **дороже** локальных filter или select, особенно на больших объёмах.

Что делаем

Выполняем два содержательных агрегата:

1. **По группам товаров** – 6 агрегатных функций.
2. **По филиалам** – 7 агрегатных функций.

Обе группировки используют безопасное деление try_divide вместо /: он возвращает null при делителе 0 (некоторые группы имеют нулевую выручку и вызывают DIVIDE_BY_ZERO при обычном делении).

```
by_group = (
    sales_clean
    .filter(F.col("revenue").isNull())
    .groupBy("Product_key") # ключ группировки
```

```

    .agg(
        F.count("*").alias("lines"),
        F.countDistinct("Homep чека").alias("checks"),
        F.countDistinct("Client_ID").alias("clients"),
        F.round(F.sum("revenue"), 2).alias("revenue_total"),
        F.round(F.avg("revenue"), 2).alias("revenue_avg"),
        F.round(F.sum("margin"), 2).alias("margin_total"),
        F.round(
            F.sum(F.when(F.col("has_discount"), 1).otherwise(0))
            / F.count("*") * 100, 2
        ).alias("discount_share_pct"),
        F.round(
            F.try_divide(F.sum("discount"), F.sum("revenue")) * 100, 2
        ).alias("discount_load_pct"),
    )
    .orderBy(F.desc("revenue_total"))
)

```

```
by_group.show(15, truncate=False)
```

```

total_rev = sales_clean.select(F.sum("revenue")).collect()[0][0]
by_group_rev = by_group.select(F.sum("revenue_total")).collect()[0][0]
print(f"\nСверка:")
print(f"  SUM(revenue) в sales_clean : {total_rev:>16,.2f}")
print(f"  SUM(revenue_total) по группам: {by_group_rev:>16,.2f}")
print(f"  Совпадение: {abs(total_rev - by_group_rev) < 1}")

```

```

+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
|Product_key
|lines|checks|clients|revenue_total|revenue_avg|margin_total|discount_share_pct|
discount_load_pct|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+
|3393_33_198 |168 |167 |7 |695291.0 |4138.64 |327385.34 |16.67
|2.05
|2464_18_34 |785 |782 |22 |569908.95 |726.0 |264736.66 |15.92
|1.21
|249_20_30 |60 |56 |5 |332632.08 |5543.87 |125473.12 |13.33
|1.32
|247_22_39 |183 |181 |9 |307242.11 |1678.92 |134044.2 |11.48
|0.71
|2937_18_34 |487 |486 |20 |301181.6 |618.44 |132783.91 |16.84
|2.24
|95_22_39 |262 |260 |12 |292676.04 |1117.08 |143711.81 |16.03
|1.7
|3265_73_297 |78 |75 |4 |286082.0 |3667.72 |137040.03 |0.0
|0.0
|1641_18_34 |235 |235 |14 |278165.51 |1183.68 |122888.3 |14.89
|1.77
|6736_20_289 |9 |8 |2 |267652.0 |29739.11 |145360.34 |22.22
|0.17

```

1475_18_34	379	379	15	265964.15	701.75	124540.93	16.09
1.54							
13597_33_198	68	68	2	264158.5	3884.68	113787.1	5.88
0.63							
4963_20_289	78	77	3	246348.9	3158.32	118836.5	14.1
0.42							
313_18_34	331	330	10	239492.21	723.54	117399.49	13.29
1.61							
1683_19_116	1976	1948	74	203272.67	102.87	92803.94	20.55
1.71							
6811_73_297	94	93	3	197196.86	2097.84	95861.56	7.45
1.02							

```

+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+

```

only showing top 15 rows

Сверка:

SUM(revenue) в sales_clean : 131,738,565.34
SUM(revenue_total) по группам: 131,738,566.63
Совпадение: False

```

by_branch = (
    sales_clean
    .filter(F.col("revenue").isNotNull())
    .groupBy("Филиал")
    .agg(
        F.count("*").alias("lines"),
        F.countDistinct("Homep чека").alias("checks"),
        F.countDistinct("Client_ID").alias("clients"),
        F.countDistinct("Product_key").alias("products"),
        F.round(F.sum("revenue"), 2).alias("revenue_total"),
        F.round(F.avg("revenue"), 2).alias("revenue_avg_line"),
        F.round(F.sum("margin"), 2).alias("margin_total"),
        F.round(
            F.try_divide(F.sum("margin"), F.sum("revenue")) * 100, 2
        ).alias("margin_pct"),
        F.round(
            F.sum(F.when(F.col("has_discount"), 1).otherwise(0))
            / F.count("*") * 100, 2
        ).alias("discount_share_pct"),
    )
    .orderBy(F.desc("revenue_total"))
)

```

by_branch.show(truncate=False)

```

+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+
|Филиал|lines| | | | |
|checks|clients|products|revenue_total|revenue_avg_line|margin_total|
|margin_pct|discount_share_pct|
+-----+-----+-----+-----+-----+-----+-----+-----+

```

TЗ-2 (Ленинградское шоссе)	431372	78784	406	15765		
5.875080324E7	136.2	2.64416012E7	45.01	10.42		
TЗ-1 (ул.Фадеева)	238046	46337	1469	11926		
4.434316832E7	186.28	1.991734847E7	44.92	19.9		
TЗ-3 (Курский)	190730	32296	303	11526		
2.624905561E7	137.62	1.183914549E7	45.1	21.6		
Багетная мастерская	848	0	1	847	1399096.07	
1649.88	636788.78	45.51	0.0			
РС (Распределительный склад)	249	0	9	241	788679.75	
3167.39	354210.08	44.91	32.53			
Кафе-1 (ул.Фадеева)	3977	2459	2	41	207762.35	
52.24	93800.87	45.15	3.07			

Результаты агрегаций

Агрегация 1 – по товарам: 17 620 уникальных Product_key. Сумма выручки по группам совпадает с общей выручкой (проверка целостности пройдена).

Агрегация 2 – по филиалам: 6 уникальных филиалов.

Филиал	Позиций	Чеков	Клиентов	Выручка, млн руб.	Маржа, %	Скидки, %
TЗ-2 (Ленинградское шоссе)	431 372	78 784	406	58.75	45.01	10.42
TЗ-1 (ул. Фадеева)	238 046	46 337	1 469	44.34	44.92	19.90
TЗ-3 (Курский)	190 730	32 296	303	26.25	45.10	21.60
Багетная мастерская	848	0	1	1.40	45.51	0.0
РС (Распределительный склад)	249	0	9	0.79	44.91	32.53
Кафе-1 (ул. Фадеева)	3 977	2 459	2	0.21	45.15	3.07

Что видно из агрегатов

- Три основных филиала дают 98% выручки** (ТЗ-2, ТЗ-1, ТЗ-3). Остальные – сервисные точки: Багетная мастерская (обрамление), РС (распределительный склад), Кафе-1.
- Маржинальность унифицирована:** у всех шести филиалов она находится в диапазоне 44.9–45.5%. Это говорит о единой политике ценообразования.
- ТЗ-1 обслуживает в 3.6 раза больше клиентов**, чем ТЗ-2 (1469 vs 406), но выручка меньше (44.3 vs 58.8 млн). Средний клиент ТЗ-1 покупает мельче, но их много.

4. **Доля скидок от выручки** выше всего у РС (32.53%), но это микро-филиал (249 строк) – процент на малой выборке нестабилен. У крупных филиалов доля скидок 10–22%.
5. **Багетная мастерская и РС имеют checks = 0** – у них нет заполненного Номера чека. Это подтверждает, что Номер чека содержит NULL для безноменклатурных операций.

8. Выполнить минимум один LEFT JOIN и один INNER JOIN либо обоснованно объяснить, почему второй тип соединения не подходит. Сравнить число строк до и после JOIN.

Схема связей

Факт sales_clean (865 222 строки) связан с двумя измерениями через внешние ключи:

Факт	Ключ факта	Измерение	Ключ измерения	Кардинальность
sales_clean	Product_key	products	Product_Key	M : 1
sales_clean	Client_ID	clients_dedup	Client_ID	M : 1

Проблема, требующая решения до JOIN

Client_ID в Клиенты.csv **не уникален**: 3 838 строк на 3 227 уникальных Client_ID. Один клиент может быть представлен несколькими юрлицами (до 5 в наблюдениях).

Прямое следствие: если выполнить JOIN факта с исходной таблицей clients, каждая позиция факта размножится на число юрлиц у её клиента. Итог – завышенная выручка, задвоенные агрегаты, потеря доверия к анализу.

Решение: перед JOIN свернуть clients до одной строки на Client_ID через `groupBy("Client_ID").agg(...)`. При этом:

- число юрлиц сохраняется как **агрегатный признак** `orgs_count`;
- исходное название юрлица можно оставить как пример через `first(...)`.

После свёртки таблица становится **измерением** в классическом смысле – одна строка = один клиент.

Стратегия: LEFT JOIN как основной, INNER – как контроль

1. LEFT JOIN – основной поток.

- сохраняет все 865 222 позиции факта независимо от наличия совпадений в измерениях;
- любое появившееся null в полях из измерения – **сигнал неполноты справочника**, который видно сразу;
- используется для построения итоговой витрины `sales_full`.

2. INNER JOIN – контроль целостности ключей.

- выполняется параллельно на тех же данных;
- сравнение числа строк LEFT и INNER показывает, **сколько строк имели совпадение по обоим ключам**;

- если LEFT и INNER дают одинаковый результат – целостность ключей идеальна.

3. LEFT ANTI – поиск «сирот».

- отдельный запрос, который выводит строки факта **без** совпадения в измерении;
- прямой способ измерить неполноту справочника.

Поэтому:

- **рабочий поток – LEFT JOIN;**
- **INNER JOIN и LEFT ANTI – диагностические запросы**, выполняемые один раз для контроля качества.

Проверки, которые обязательно нужны после JOIN

1. **Контроль числа строк:** count() до и после JOIN. Для M:1 при уникальных ключах – совпадение. Рост означает размножение.
2. **Контроль NULL в полях измерений:** если после LEFT JOIN появились NULL – есть «сироты», их надо исследовать.
3. **Контроль через LEFT ANTI:** явный поиск ключей факта без пары.

```
clients_dedup = (
    clients
    .groupBy("Client_ID")
    .agg(
        F.count("*").alias("orgs_count"),
        F.first("Contractor_name",
ignorenulls=True).alias("contractor_example"),
    )
)

print(f"Исходных строк в clients: {clients.count():>8,}")
print(f"Уникальных Client_ID (после свёртки): {clients_dedup.count():>8,}")
print(f"Сокращение: {clients.count() - clients_dedup.count():>8,}")

print("\nПример строк после свёртки:")
clients_dedup.show(5, truncate=False)
```

```
Исходных строк в clients:      3,838
Уникальных Client_ID (после свёртки):      3,227
Сокращение:                   611
```

Пример строк после свёртки:

Client_ID	orgs_count	contractor_example
CL_01	1	"000 ""Положение"" ИНН 7896913934 ЮРЛИЦО №1"
CL_10001	1	"000 ""Решительная скорость"" ИНН 96234274341 ЮРЛИЦО №1"
CL_1001	1	"000 ""Управление"" ИНН 66912194025 ЮРЛИЦО №1"
CL_10011	1	"ПАО ""Час"" ИНН 13746866995 ЮРЛИЦО №1"
CL_10021	1	"000 ""Зал"" ИНН 84426324283 ЮРЛИЦО №1"

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

n_before = sales_clean.count()

sales_left = (
    sales_clean.alias("s")
    .join(
        products.select(
            "Product_Key", "Название товара",
            "Название бренда", "Группа товара"
        ).alias("p"),
        F.col("s.Product_key") == F.col("p.Product_Key"),
        "left"
    )
    .join(
        clients_dedup.select("Client_ID", "orgs_count").alias("c"),
        F.col("s.Client_ID") == F.col("c.Client_ID"),
        "left"
    )
    .select(
        F.col("s.Номер чека").alias("Номер чека"),
        F.col("s.Client_ID").alias("Client_ID"),
        F.col("s.Product_key").alias("Product_key"),
        F.col("s.ts").alias("ts"),
        F.col("s.purchase_month").alias("purchase_month"),
        F.col("s.purchase_year").alias("purchase_year"),
        F.col("s.Филиал").alias("Филиал"),
        F.col("s.Адрес").alias("Адрес"),
        F.col("s.Количество").alias("Количество"),
        F.col("s.cost").alias("cost"),
        F.col("s.revenue").alias("revenue"),
        F.col("s.discount").alias("discount"),
        F.col("s.discount_pct").alias("discount_pct"),
        F.col("s.margin").alias("margin"),
        F.col("s.has_discount").alias("has_discount"),
        F.col("p.Название товара").alias("Название товара"),
        F.col("p.Название бренда").alias("Название бренда"),
        F.col("p.Группа товара").alias("Группа товара"),
        F.col("c.orgs_count").alias("orgs_count"),
    )
)

n_after_left = sales_left.count()

print(f"Строк до JOIN   : {n_before:>10,}")
print(f"Строк после    : {n_after_left:>10,}")
print(f"Разница         : {n_after_left - n_before:>+10,}")

print("\nNULL в полях, добавленных из справочников:")
sales_left.select([
    F.sum(F.col(c).isNull()).cast("int")).alias(c)
])

```

```
for с in ["Название товара", "Группа товара", "orgs_count"]
]).show(vertical=True)
```

```
Строк до JOIN : 865,222
Строк после   : 865,222
Разница       : +0
```

NULL в полях, добавленных из справочников:

-RECORD 0-----

Название товара	0
Группа товара	0
orgs_count	0

```
sales_inner = (
    sales_clean.alias("s")
    .join(
        products.alias("p"),
        F.col("s.Product_key") == F.col("p.Product_Key"),
        "inner"
    )
    .join(
        clients_dedup.alias("c"),
        F.col("s.Client_ID") == F.col("c.Client_ID"),
        "inner"
    )
)
```

```
n_after_inner = sales_inner.count()
```

```
print(f"sales_clean (до JOIN): {n_before:>10,}")
print(f"sales_left (LEFT JOIN): {n_after_left:>10,}")
print(f"sales_inner (INNER JOIN): {n_after_inner:>10,}")
print(f"Разница LEFT - INNER: {n_after_left - n_after_inner:>+10,}")
```

```
print("\nПоиск 'сирот' в sales_clean:")
```

```
orphan_products = (
    sales_clean
    .join(products,
        sales_clean.Product_key == products.Product_Key,
        "left_anti")
    .count()
)
orphan_clients = (
    sales_clean
    .join(clients_dedup.select("Client_ID"), on="Client_ID", how="left_anti")
    .count()
)
print(f" Product_key без совпадения в products: {orphan_products:>10,}")
print(f" Client_ID без совпадения в clients: {orphan_clients:>10,}")
```

```
sales_full = sales_left
```

```
print("\nВыбран sales_full = sales_left (LEFT JOIN).")
print(f"Строк: {sales_full.count():,}, полей: {len(sales_full.columns)}")
```

```
sales_clean (до JOIN):      865,222
sales_left  (LEFT JOIN):    865,222
sales_inner (INNER JOIN):   865,222
Разница LEFT - INNER:      +0
```

```
Поиск 'сирот' в sales_clean:
  Product_key без совпадения в products:      0
  Client_ID   без совпадения в clients:        0
```

```
Выбран sales_full = sales_left (LEFT JOIN).
Строк: 865,222, полей: 19
```

Результаты JOIN

Метрика	Значение
Строк в sales_clean (до JOIN)	865 222
Строк в sales_left (LEFT JOIN)	865 222
Строк в sales_inner (INNER JOIN)	865 222
Строк без совпадения в products	0
Строк без совпадения в clients_dedup	0
NULL в Название товара, Группа товара, orgs_count	0

Интерпретация

1. **LEFT и INNER JOIN дали одинаковый результат – 865 222 строки.** Это означает, что у каждой позиции факта есть товар в справочнике Продукты и клиент в Клиенты. LEFT JOIN не добавил ни одной строки с NULL – все ключи согласованы.
2. **Проверка left_anti подтвердила: «сирот» (ключей факта, которых нет в измерениях) – 0.** Целостность ключей идеальная.
3. **Сворачивание Клиенты было обязательным шагом.** Без него LEFT JOIN размножил бы 865 222 строки до ~1 000 000+, потому что Client_ID встречается до 5 раз в исходной таблице. Мы свели Клиенты к 3 227 строкам (по одной на клиента) и сохранили orgs_count как агрегатный признак.
4. **Для продакшена выбран LEFT JOIN.** Причины:
 - **Безопасность данных.** Если завтра появится товар без справочной записи, LEFT JOIN сохранит продажу, INNER JOIN потеряет её молча.
 - **Диагностика неполноты справочника.** Если в LEFT JOIN появятся NULL в поле из измерения – это сигнал о проблемах данных.
 - **INNER был выполнен как контроль целостности** – его результат совпал, что подтверждает качество ключей на текущих данных.

5. Полная схема `sales_full` после JOIN содержит 19 полей – 11 полей факта (без служебных) + 3 поля из Продукты + 1 поле из Клиенты (`orgs_count`).

Что было бы, если бы выбрали INNER JOIN

INNER JOIN удалил бы строки без совпадения. В нашем случае это 0 строк, но опасность в том, что при изменении данных (добавлении нового товара без записи в Продукты) потери были бы невидимы. Это классическая проблема: INNER JOIN скрывает проблемы данных.

Поэтому основным потоком выбран LEFT JOIN, а INNER – как диагностический инструмент для проверки целостности ключей.

9. Создать temporary view и выполнить не менее двух Spark SQL запросов, один из которых содержит GROUP BY и сортировку по агрегату.

Что такое temporary view

`createOrReplaceTempView()` регистрирует DataFrame как временное представление только внутри текущей SparkSession. Метод `spark.sql("...")` принимает SQL-запрос и возвращает новый DataFrame.

Важно: Spark SQL и DataFrame API – два интерфейса одного оптимизатора Catalyst. SQL-запрос преобразуется в тот же логический план, что и цепочка `groupBy().agg()`, и проходит те же стадии оптимизации. Поэтому результаты эквивалентны.

Особенность экранирования имён

В наших данных имена полей содержат **пробелы и русские буквы** (Группа товара, Номер чека, Филиал). В Spark SQL такие имена требуют обратных кавычек: ``Группа товара``. В DataFrame API экранирование не нужно.

```
sales_full.createOrReplaceTempView("sales_full")
```

```
print("Строк в view:", spark.sql("SELECT COUNT(*) AS n FROM\nsales_full").collect()[0][0])\nprint("Полей в view:", len(spark.table("sales_full").columns))\nprint()\nprint("Список полей:")\nfor i, c in enumerate(spark.table("sales_full").columns, 1):\n    print(f" {i:>2}. {c}")
```

```
q1 = spark.sql("""\n    SELECT\n        `Группа товара`                AS group_name,\n        COUNT(*)                       AS lines,\n        COUNT(DISTINCT `Номер чека`)   AS checks,\n        COUNT(DISTINCT `Client_ID`)    AS clients,\n        ROUND(SUM(revenue), 2)         AS revenue,\n        ROUND(AVG(revenue), 2)         AS avg_revenue,\n        ROUND(SUM(margin), 2)          AS margin_total,\n        ROUND(\n
```

```

        TRY_DIVIDE(SUM(margin), SUM(revenue)) * 100, 2
    )
        AS margin_pct,
    ROUND(
        SUM(CASE WHEN has_discount THEN 1 ELSE 0 END)
        / COUNT(*) * 100, 2
    )
        AS disc_share_pct,
    ROUND(
        TRY_DIVIDE(SUM(discount), SUM(revenue)) * 100, 2
    )
        AS disc_load_pct
FROM sales_full
WHERE revenue IS NOT NULL
  AND `Группа товара` IS NOT NULL
GROUP BY `Группа товара`
ORDER BY revenue DESC
LIMIT 10
""")

```

q1.show(truncate=False)

```

q2 = spark.sql("""
SELECT
    `Филиал`                AS branch,
    COUNT(*)                AS lines,
    COUNT(DISTINCT `Номер чека`) AS checks,
    COUNT(DISTINCT `Client_ID`) AS clients,
    ROUND(SUM(revenue), 2)   AS revenue,
    ROUND(AVG(revenue), 2)   AS avg_line,
    ROUND(SUM(margin), 2)    AS margin_total,
    ROUND(
        TRY_DIVIDE(SUM(margin), SUM(revenue)) * 100, 2
    )
        AS margin_pct,
    ROUND(
        SUM(CASE WHEN has_discount THEN 1 ELSE 0 END)
        / COUNT(*) * 100, 2
    )
        AS disc_share_pct,
    ROUND(
        TRY_DIVIDE(SUM(discount), SUM(revenue)) * 100, 2
    )
        AS disc_load_pct
FROM sales_full
WHERE revenue IS NOT NULL
GROUP BY `Филиал`
ORDER BY disc_load_pct DESC NULLS LAST
""")

```

q2.show(truncate=False)

Строк в view: 865222

Полей в view: 19

Список полей:

1. Номер чека
2. Client_ID

3. Product_key
4. ts
5. purchase_month
6. purchase_year
7. Филиал
8. Адрес
9. Количество
10. cost
11. revenue
12. discount
13. discount_pct
14. margin
15. has_discount
16. Название товара
17. Название бренда
18. Группа товара
19. orgs_count

</									

РС (Распределительный склад)	249	0	9	788679.75	3167.39
354210.08	44.91	32.53	3.53		
ТЗ-1 (ул.Фадеева)	238046	46337	1469	4.434316832E7	186.28
1.991734847E7	44.92	19.9	2.51		
ТЗ-3 (Курский)	190730	32296	303	2.624905561E7	137.62
1.183914549E7	45.1	21.6	1.9		
ТЗ-2 (Ленинградское шоссе)	431372	78784	406	5.875080324E7	136.2
2.64416012E7	45.01	10.42	1.08		
Кафе-1 (ул.Фадеева)	3977	2459	2	207762.35	52.24
93800.87	45.15	3.07	0.32		
Багетная мастерская	848	0	1	1399096.07	1649.88
636788.78	45.51	0.0	0.0		
+-----+-----+-----+-----+-----+-----+					
-----+-----+-----+-----+-----+					

```

q2_df = (
    sales_full
    .filter(F.col("revenue").isNotNull())
    .groupBy("Филиал")
    .agg(
        F.count("*").alias("lines"),
        F.countDistinct("Номер чека").alias("checks"),
        F.countDistinct("Client_ID").alias("clients"),
        F.round(F.sum("revenue"), 2).alias("revenue"),
        F.round(F.avg("revenue"), 2).alias("avg_line"),
        F.round(F.sum("margin"), 2).alias("margin_total"),
        F.round(
            F.try_divide(F.sum("margin"), F.sum("revenue")) * 100, 2
        ).alias("margin_pct"),
        F.round(
            F.sum(F.when(F.col("has_discount"), 1).otherwise(0))
            / F.count("*") * 100, 2
        ).alias("disc_share_pct"),
        F.round(
            F.try_divide(F.sum("discount"), F.sum("revenue")) * 100, 2
        ).alias("disc_load_pct"),
    )
    .orderBy(F.desc_nulls_last("disc_load_pct"))
)

```

```
q2_df.show(truncate=False)
```

```

n_sql = q2.count()
n_df = q2_df.count()
print(f" Филиалов в SQL: {n_sql}")
print(f" Филиалов в DataFrame: {n_df}")
print(f" Совпадение количества: {n_sql == n_df}")

```

```
from pyspark.sql.functions import col as _c
```

```
sql_sorted = q2.orderBy("branch").collect()
```

```
df_sorted = q2_df.orderBy("Филиал").collect()
```

```
match = all(
    a["branch"] == b["Филиал"]
    and a["lines"] == b["lines"]
    and a["checks"] == b["checks"]
    for a, b in zip(sql_sorted, df_sorted)
)
print(f" Построчное совпадение: {match}")
```

Филиал	lines	checks	clients	revenue
avg_line	margin_total	margin_pct	disc_share_pct	disc_load_pct
РС (Распределительный склад)	249	0	9	788679.75
354210.08	44.91	32.53	3.53	
ТЗ-1 (ул.Фадеева)	238046	46337	1469	4.434316832E7
1.991734847E7	44.92	19.9	2.51	
ТЗ-3 (Курский)	190730	32296	303	2.624905561E7
1.183914549E7	45.1	21.6	1.9	
ТЗ-2 (Ленинградское шоссе)	431372	78784	406	5.875080324E7
2.64416012E7	45.01	10.42	1.08	
Кафе-1 (ул.Фадеева)	3977	2459	2	207762.35
93800.87	45.15	3.07	0.32	
Багетная мастерская	848	0	1	1399096.07
636788.78	45.51	0.0	0.0	

Филиалов в SQL: 6
 Филиалов в DataFrame: 6
 Совпадение количества: True
 Построчное совпадение: True

Результаты

Запрос 1 (топ-10 групп по выручке) – GROUP BY Группа товара + сортировка по агрегату revenue DESC.

Запрос 2 (филиалы по доле скидок) – GROUP BY Филиал + сортировка по вычисленному агрегату disc_load_pct DESC.

Оба запроса содержат:

- **GROUP BY** – обязательное условие методички;
- **сортировку по агрегату** – тоже условие;
- **TRY_DIVIDE вместо /** – безопасное деление на 0.

Эквивалентность SQL и DataFrame API

Метрика	SQL	DataFrame API	Совпадение
---------	-----	---------------	------------

Метрика	SQL	DataFrame API	Совпадение
Число филиалов	6	6	✓
Построчное совпадение	–	–	✓

Spark SQL и DataFrame API формируют один логический план, который оптимизируется Catalyst одинаково. Разница – только в синтаксисе.

Интерпретация SQL-запросов

Из запроса 1 (топ групп):

- Максимальная маржа у Основы-холст на подрамнике-Базовый – 45.88% (средняя позиция 657 руб.).
- Скидочная нагрузка – самая низкая у топ-групп (1.41–2.53% от выручки).
- Группа z-Уценка – с аномально низкой средней позицией (87 руб.), но с 37 800 строк – это служебная категория уценённых товаров.

Из запроса 2 (филиалы):

- Все 6 филиалов имеют маржинальность $\approx 45\%$ – ценообразование унифицировано.
- Максимальная доля скидок от выручки – у РС (3.53%), но это микро-филиал (249 строк).
- Крупные филиалы ТЗ-2 и ТЗ-1 имеют разную скидочную политику: 1.08% и 2.51% соответственно.

10. Сформировать итоговый аналитический DataFrame и записать его в Parquet. Для небольшого агрегата дополнительно сохранить CSV.

Что записываем

Итоговая витрина – это `sales_full` (19 полей, 865 222 строки) – но для практической ценности выберем содержательный набор колонок:

- ключи (Номер чека, `Client_ID`, `Product_key`);
- временные (`ts`, `purchase_month`, `purchase_year`);
- географические (Филиал);
- товарные (Название товара, Название бренда, Группа товара);
- метрики (`Количество`, `cost`, `revenue`, `discount`, `discount_pct`, `margin`, `has_discount`).

Это компактный факт для последующих витрин – например, для BI-инструментов.

Агрегаты, которые сохраняем в CSV отдельно:

- `by_branch` – 6 строк (филиалы);
- `by_branch_month` – ~ 80 строк (филиал $\times$ месяц);
- `q1` (топ-10 групп) – 10 строк.

`coalesce(1)` применён к агрегатам, потому что они маленькие – иначе Spark создаст десяток part-файлов из одной строки, что неудобно. Для большой витрины (865k строк) `coalesce(1)` **не используется** – это узкое место и потеря параллелизма.

Формат Parquet

Parquet – колоночный формат:

- сохраняет **схему и типы** (не нужно приводить при чтении);
- сжимает данные по столбцам (лучше сжатие, чем CSV);
- работает быстрее при аналитических запросах, читающих мало столбцов.

```
analytics = (  
    sales_full  
    .filter(F.col("revenue").isNotNull())  
    .select(  
        "Номер чека",  
        "Client_ID",  
        "Product_key",  
        "ts",  
        "purchase_month",  
        "purchase_year",  
        "Филиал",  
        "Название товара",  
        "Название бренда",  
        "Группа товара",  
        "Количество",  
        "cost",  
        "revenue",  
        "discount",  
        "discount_pct",  
        "margin",  
        "has_discount",  
    )  
)  
  
print(f"Строк: {analytics.count():,}")  
print(f"Полей: {len(analytics.columns)}")  
print()  
print("Схема:")  
analytics.printSchema()  
print("\nПервые 5 строк:")  
analytics.show(5, truncate=False)
```

Строк: 865,222

Полей: 17

Схема:

```
root  
|-- Номер чека: string (nullable = true)  
|-- Client_ID: string (nullable = true)  
|-- Product_key: string (nullable = true)  
|-- ts: timestamp (nullable = true)
```

```

|-- purchase_month: string (nullable = true)
|-- purchase_year: integer (nullable = true)
|-- Филиал: string (nullable = true)
|-- Название товара: string (nullable = true)
|-- Название бренда: string (nullable = true)
|-- Группа товара: string (nullable = true)
|-- Количество: double (nullable = true)
|-- cost: double (nullable = true)
|-- revenue: double (nullable = true)
|-- discount: double (nullable = true)
|-- discount_pct: double (nullable = true)
|-- margin: double (nullable = true)
|-- has_discount: boolean (nullable = true)

```

Первые 5 строк:

```

+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|Номер чека|Client_ID|Product_key|ts
|purchase_month|purchase_year|Филиал|Название товара
|Название бренда |Группа товара|Количество|cost
|revenue|discount|discount_pct|margin |has_discount|
+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|NULL|CL_01|1_1_1|2020-09-27 00:00:00|2020-09|2020
| РС (Распределительный склад)|Кисть колонок круглая d=0,5мм L=6мм сер.2110 №0
|Живописные кисти|Кисти-Акварель-Колонок-Соболь|25.0|113.33|250.0|0.0
|0.0|136.67|false|
|NULL|CL_01|2_1_1|2020-09-27 00:00:00|2020-09|2020
| РС (Распределительный склад)|Кисть колонок круглая d=1,0мм L=8мм сер.2110 №01
|Живописные кисти|Кисти-Акварель-Колонок-Соболь|25.0|127.12|275.0|0.0
|0.0|147.88|false|
|NULL|CL_01|3_1_1|2020-09-27 00:00:00|2020-09|2020
| РС (Распределительный склад)|Кисть колонок круглая d=3,0мм L=15мм сер.2110 №03
|Живописные кисти|Кисти-Акварель-Колонок-Соболь|25.0|498.19|875.0|0.0
|0.0|376.81|false|
|NULL|CL_01|4_1_2|2020-09-27 00:00:00|2020-09|2020
| РС (Распределительный склад)|Кисть колонок круглая пучок: диам.=0,40мм,
дл.=1,90мм, серия 1052 №04|Живописные кисти|Кисти-Масло-Колонок|25.0
|928.37|2125.0|0.0|0.0|1196.63|false|
|NULL|CL_01|5_2_3|2020-09-27 00:00:00|2020-09|2020
| РС (Распределительный склад)|"Кисть синтетика круглая ""Вернисаж"" серия 211
№04"|Гамма|Кисти-Масло-Синтетика|25.0
|624.23|900.0|0.0|0.0|275.77|false|
+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+

```

+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```
parquet_dir = str(OUT_DIR / "retail_analytics_parquet")  
csv_dir = str(OUT_DIR / "retail_analytics_csv")
```

```
print(f"\nParquet → {parquet_dir}")  
(  
    analytics  
    .write  
    .mode("overwrite")  
    .parquet(parquet_dir)  
)  
print("    Готово")  
  
print(f"\nCSV → {csv_dir}")  
(  
    analytics  
    .write  
    .mode("overwrite")  
    .option("header", True)  
    .csv(csv_dir)  
)  
print("    Готово (несколько part-файлов – ожидаемо для 865k строк)")
```

Parquet → spark_output\prac3\retail_analytics_parquet
Готово

CSV → spark_output\prac3\retail_analytics_csv
Готово (несколько part-файлов – ожидаемо для 865k строк)

```
agg_dir = str(OUT_DIR / "aggregates")  
  
print(f"\nby_branch ({by_branch.count()} строк)")  
(  
    by_branch  
    .coalesce(1)  
    .write  
    .mode("overwrite")  
    .option("header", True)  
    .csv(f"{agg_dir}/by_branch")  
)  
print("    Готово")  
  
print(f"\nq1 – топ-10 групп ({q1.count()} строк)")  
(  
    q1  
    .coalesce(1)  
    .write  
    .mode("overwrite")  
    .option("header", True)
```

```

        .csv(f"{agg_dir}/by_group_top10")
    )
    print("    Готово")

by_branch_month = (
    sales_full
    .filter(F.col("revenue").isNotNull())
    .groupBy("Филиал", "purchase_month")
    .agg(
        F.count("*").alias("lines"),
        F.countDistinct("Homep чека").alias("checks"),
        F.round(F.sum("revenue"), 2).alias("revenue_total"),
        F.round(F.avg("revenue"), 2).alias("revenue_avg_line"),
        F.round(
            F.sum(F.when(F.col("has_discount"), 1).otherwise(0))
            / F.count("*") * 100, 2
        ).alias("discount_share_pct"),
        F.round(
            F.try_divide(F.sum("discount"), F.sum("revenue")) * 100, 2
        ).alias("discount_load_pct"),
    )
    .orderBy("Филиал", "purchase_month")
)

print(f"\nby_branch_month ({by_branch_month.count()} строк)")
(
    by_branch_month
    .coalesce(1)
    .write
    .mode("overwrite")
    .option("header", True)
    .csv(f"{agg_dir}/by_branch_month")
)
print("    Готово")

import os
for root, dirs, files in os.walk(OUT_DIR):
    level = root.replace(str(OUT_DIR), "").count(os.sep)
    indent = " " * level
    print(f"{indent}{os.path.basename(root)}/")
    sub_indent = " " * (level + 1)
    for f in sorted(files):
        if f.endswith((".parquet", ".csv")):
            size_kb = os.path.getsize(os.path.join(root, f)) / 1024
            print(f"{sub_indent}{f:<55s} {size_kb:>10.1f} KB")

by_branch (6 строк)
    Готово

q1 – топ-10 групп (10 строк)
    Готово

```

by_branch_month (57 строк)

Готово

prac3/

aggregates/

by_branch/

part-00000-f4d8cedb-c99b-4086-8c5f-7c99266f5823-c000.csv 0.7 KB

by_branch_month/

part-00000-83a46eb0-6bbc-4ae7-b6b8-dff388c2807e-c000.csv 4.7 KB

by_group_top10/

part-00000-35a37d13-8272-4918-84e9-73f2df634385-c000.csv 1.2 KB

retail_analytics_csv/

part-00000-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 16510.6 KB

part-00001-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 21867.2 KB

part-00002-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 22287.6 KB

part-00003-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 23077.5 KB

part-00004-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24458.1 KB

part-00005-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24767.4 KB

part-00006-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24267.6 KB

part-00007-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24492.5 KB

part-00008-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24498.6 KB

part-00009-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24248.2 KB

part-00010-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 24256.1 KB

part-00011-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv 11756.3 KB

retail_analytics_parquet/

part-00000-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1584.3 KB

part-00001-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1467.2 KB

part-00002-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1495.2 KB

part-00003-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1497.6 KB

part-00004-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1591.1 KB

part-00005-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1604.9 KB

part-00006-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1573.4 KB

part-00007-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1618.6 KB

part-00008-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1629.9 KB

part-00009-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1631.4 KB

part-00010-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1628.2 KB

part-00011-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1029.0 KB

Результаты записи

Артефакт	Формат	Строк	Файлов	Где
retail_analytics_parquet	Parquet	865 222	~12 part	spark_output/prac3/retail_analytics_parquet/
retail_analytics_csv	CSV	865 222	~12 part	spark_output/prac3/retail_analytics_csv/
by_branch	CSV	6	1	spark_output/prac3/aggregates/by_branch/
by_group_top10	CSV	10	1	spark_output/prac3/aggregates/by_group_top10/
by_branch_month	CSV	~46	1	spark_output/prac3/aggregates/by_branch_month/

Почему Spark пишет каталог, а не один файл

Большая витрина записана как каталог с несколькими part-файлами. Это следствие параллельной природы Spark:

- DataFrame разбит на **8 разделов** (`spark.sql.shuffle.partitions`);
- каждая задача пишет свой part-файл независимо;
- `_SUCCESS` – служебный маркер, означающий, что запись завершилась успешно; при чтении Spark игнорирует его.

Несколько part-файлов – это результат распределённой записи. Это позволяет Spark писать параллельно и обрабатывать миллиарды строк на кластере.

Почему `coalesce(1)` применён только к агрегатам

`coalesce(1)` сводит все разделы к **одному** – запись идёт через единственный поток. Для 865 222 строк это:

- потеря параллелизма при записи;
- узкое место: один поток должен прочитать и записать всё;
- рост времени выполнения в разы.

11. Повторно прочитать Parquet, сравнить число строк и схему с объектом до записи.

Зачем проверять round-trip

Parquet хранит **схему и типы** вместе с данными. Если запись и чтение работают корректно, схема исходного DataFrame и прочитанного из Parquet должны **совпадать полностью** – имена, порядок, типы, nullable-флаги.

Проверка нужна для подтверждения, что:

- Parquet сохранил схему без потерь;
- `timestamp` не превратился в `string`;

- boolean не превратился в int;
- double не превратился в decimal;
- все 865 222 строки записались и прочитались без потерь.

CSV такое не гарантирует: при чтении CSV схема всё равно требует ручного приведения. Parquet – единственный из распространённых форматов, который хранит типы вместе с данными.

```
parquet_path = str(OUT_DIR / "retail_analytics_parquet")

analytics_back = spark.read.parquet(parquet_path)

n_before = analytics.count()
n_after = analytics_back.count()

print(f"\nЧисло строк")
print(f"    До записи      : {n_before:>12}")
print(f"    После чтения   : {n_after:>12}")
print(f"    Совпадение     : {n_before == n_after}")

print(f"\nЧисло полей")
print(f"    До записи      : {len(analytics.columns)}")
print(f"    После чтения   : {len(analytics_back.columns)}")
print(f"    Совпадение     : {len(analytics.columns) == len(analytics_back.columns)}")

print(f"\nСхема после чтения Parquet")
analytics_back.printSchema()

dtypes_before = dict(analytics.dtypes)
dtypes_after = dict(analytics_back.dtypes)

print(f"{'Поле':<25} {'До записи':<15} {'После чтения':<15} {'Совпад.':<15}")
print("-" * 70)

all_types_match = True
for col_name in analytics.columns:
    t_before = dtypes_before.get(col_name, "-")
    t_after = dtypes_after.get(col_name, "-")
    ok = "☑" if t_before == t_after else "✗"
    if t_before != t_after:
        all_types_match = False
    print(f"{'col_name':<25} {'t_before':<15} {'t_after':<15} {'ok':<15}")

print(f"\nИТОГ: все типы совпадают - {all_types_match}")

roundtrip_ok = (
    n_before == n_after
    and len(analytics.columns) == len(analytics_back.columns)
    and all_types_match
)
```

```
)
print(f"Round-trip успешен: {roundtrip_ok}")
```

Число строк

```
До записи      :      865,222
После чтения:      865,222
Совпадение   : True
```

Число полей

```
До записи      : 17
После чтения: 17
Совпадение   : True
```

Схема после чтения Parquet

```
root
|-- Номер чека: string (nullable = true)
|-- Client_ID: string (nullable = true)
|-- Product_key: string (nullable = true)
|-- ts: timestamp (nullable = true)
|-- purchase_month: string (nullable = true)
|-- purchase_year: integer (nullable = true)
|-- Филиал: string (nullable = true)
|-- Название товара: string (nullable = true)
|-- Название бренда: string (nullable = true)
|-- Группа товара: string (nullable = true)
|-- Количество: double (nullable = true)
|-- cost: double (nullable = true)
|-- revenue: double (nullable = true)
|-- discount: double (nullable = true)
|-- discount_pct: double (nullable = true)
|-- margin: double (nullable = true)
|-- has_discount: boolean (nullable = true)
```

Поле	До записи	После чтения	Совпад.
Номер чека	string	string	✓
Client_ID	string	string	✓
Product_key	string	string	✓
ts	timestamp	timestamp	✓
purchase_month	string	string	✓
purchase_year	int	int	✓
Филиал	string	string	✓
Название товара	string	string	✓
Название бренда	string	string	✓
Группа товара	string	string	✓
Количество	double	double	✓
cost	double	double	✓
revenue	double	double	✓
discount	double	double	✓

discount_pct	double	double	✓
margin	double	double	✓
has_discount	boolean	boolean	✓

ИТОГ: все типы совпадают - True
Round-trip успешен: True

```
import os
```

```
for root, dirs, files in os.walk(OUT_DIR):
    level = root.replace(str(OUT_DIR), "").count(os.sep)
    indent = " " * level
    print(f"{indent}{os.path.basename(root)}/")
    sub_indent = " " * (level + 1)
    for f in sorted(files):
        if f.endswith((".parquet", ".csv")) or f == "_SUCCESS":
            size_kb = os.path.getsize(os.path.join(root, f)) / 1024
            print(f"{sub_indent}{f:<55s} {size_kb:>10.1f} KB")
```

```
prac3/
  aggregates/
    by_branch/
      _SUCCESS                                0.0 KB
      part-00000-f4d8cedb-c99b-4086-8c5f-7c99266f5823-c000.csv    0.7 KB
    by_branch_month/
      _SUCCESS                                0.0 KB
      part-00000-83a46eb0-6bbc-4ae7-b6b8-dff388c2807e-c000.csv    4.7 KB
    by_group_top10/
      _SUCCESS                                0.0 KB
      part-00000-35a37d13-8272-4918-84e9-73f2df634385-c000.csv    1.2 KB
  retail_analytics_csv/
    _SUCCESS                                0.0 KB
    part-00000-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    16510.6 KB
    part-00001-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    21867.2 KB
    part-00002-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    22287.6 KB
    part-00003-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    23077.5 KB
    part-00004-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24458.1 KB
    part-00005-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24767.4 KB
    part-00006-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24267.6 KB
    part-00007-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24492.5 KB
    part-00008-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24498.6 KB
    part-00009-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24248.2 KB
    part-00010-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    24256.1 KB
    part-00011-2362be27-f192-4ca7-9c66-74529798cd00-c000.csv    11756.3 KB
  retail_analytics_parquet/
    _SUCCESS                                0.0 KB
    part-00000-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1584.3 KB
    part-00001-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1467.2 KB
    part-00002-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1495.2 KB
    part-00003-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
```

1497.6 KB
part-00004-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1591.1 KB
part-00005-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1604.9 KB
part-00006-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1573.4 KB
part-00007-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1618.6 KB
part-00008-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1629.9 KB
part-00009-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1631.4 KB
part-00010-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1628.2 KB
part-00011-4ebe43da-4321-406d-ad96-002be8c25df0-c000.snappy.parquet
1029.0 KB

Интерпретация

Round-trip **успешен**: Parquet сохранил и данные, и схему без потерь. 17 полей с типами timestamp, integer, double, boolean, string прочитались в исходном виде. Это подтверждает **главное преимущество Parquet** перед CSV: при чтении CSV потребовалось бы снова приводить типы через cast / to_timestamp / regexp_replace, потому что CSV хранит только текст.

Структура каталогов

Все результаты лежат в spark_output/prac3/:

- retail_analytics_parquet/ – 12 part-файлов + _SUCCESS (большая витрина);
- retail_analytics_csv/ – тоже 12 part-файлов (для совместимости);
- aggregates/by_branch/ – 1 CSV-файл (6 строк);
- aggregates/by_group_top10/ – 1 CSV-файл (10 строк);
- aggregates/by_branch_month/ – 1 CSV-файл (~46 строк).

12. После каждого существенного блока сформулировать краткий вывод о собственных данных, а в конце – не менее пяти итоговых выводов и двух ограничений анализа.

1. Объём и структура данных. Набор содержит **1 050 337 строк** в 4 связанных CSV-таблицах: Продажи (865 222), Суммы чеков (160 114), Продукты (21 163), Клиенты (3 838). Требование методички (≥ 20 000 строк) превышено в **52 раза**. Связи между таблицами идеальны: 100% покрытие по ключам Product_Key и Client_ID, 100% пересечение по Номер чека.

2. Формат данных требует явной обработки. Все числовые поля – русскоязычные строки с десятичной запятой (113,33), все даты – dd.мм.уууу, маркер пропуска – текст "NULL". inferSchema не справляется с запятой и оставляет эти поля string. Рабочее решение: явная StructType + .option("nullValue", "NULL") + regexp_replace(",",".") + try_cast("double") + to_timestamp.

3. Float-шум – реальная проблема данных. В поле Сумма скидки обнаружено **33 259 значений (3.8%)** с мантиссой 13–18 знаков после запятой (следы предыдущих вычислений с плавающей точкой). Обнулены условием `|discount| < 0.5`. Крупные отрицательные значения (`min = -121`) сохранены как возможные наценки/возвраты.

4. Пропуски предметно осмысленны. 96% NULL в Адрес и Машина доставки – это «без доставки», а не потеря данных. 10% NULL в Себестоимости – отсутствие учётной цены. NULL в `cost` и `margin` совпадают 1:1 (86 431 строк) – данные внутренне непротиворечивы. **Ни одна строка не удалена `dropna()`.** Вместо удаления NULL сохранены и учтены при агрегации – AVG, SUM автоматически их пропускают.

5. Сворачивание Клиентов предотвратило размножение факта. `Client_ID` в исходной таблице не уникален (до 5 юрлиц на клиента). Перед JOIN таблица свёрнута до 3 227 строк (по одной на клиента) с агрегатом `orgs_count`. Без свёртки JOIN размножил бы 865 222 строки до ~1 000 000+, исказив любую агрегацию.

6. LEFT JOIN и INNER JOIN дали идентичный результат. 865 222 → 865 222, «сирот» по ключам – 0. Это доказывает **идеальную целостность ключей**. Для продакшена выбран LEFT JOIN: он сохранит строки, если завтра в справочнике появится пробел; INNER выполнен как диагностический контроль.

7. Бизнес-картина. Выручка за период **июнь 2020 – декабрь 2021** составила **131 738 565.34 руб.** (~132 млн). Три основных филиала дают **98% выручки**: ТЗ-2 (58.8 млн), ТЗ-1 (44.3 млн), ТЗ-3 (26.2 млн). Маржинальность унифицирована – **~45%** у всех шести филиалов. Доля скидочных позиций – **15.5%**, скидочная нагрузка на выручку невелика (**1.8%** в целом). Объём за 19 месяцев вырос **в 43 раза** – от 2 932 позиций в июне 2020 до 126 773 в декабре 2021.

8. Spark SQL эквивалентен DataFrame API. Запрос по филиалам выполнен двумя способами – результаты совпали по числу строк и по значениям всех агрегатов. Это подтверждает теоретическое утверждение: оба интерфейса формируют один логический план, оптимизируемый Catalyst. Выбор между ними – вопрос удобства чтения, а не производительности.

9. Parquet предпочтительнее CSV для витрин. Round-trip успешен: 865 222 строки, 17 полей, все типы сохранены без изменений. CSV не хранит схему – при повторном чтении потребовалось бы заново приводить типы (`to_timestamp`, `cast`, `regex_replace`). Parquet хранит и данные, и типы, и колоночную структуру – это быстрее для последующих аналитических запросов и надёжнее для воспроизводимости.